Administrivia

Thank you all for the feedback!

Quiz at end of class

HW4 progress check

HW5 out for the early birds

Traits

Define a set of methods that a type must implement

Like an interface in Java

Trait example

```
trait Animal {
    fn new(name: String) -> Self;
    fn say_hi(&self);
}
```

```
struct Cow {
    name: String,
}
```

```
impl Animal for Cow {
    fn new(name: String) -> Self {
        Cow {name}
    }
    fn say_hi(&self) {
        println!("cow named {self.name}");
    }
}
```

Trait example

```
trait Animal {
    fn new(name: String) -> Self;
    fn say_hi(&self);
}
```

```
struct Dog {
    name: String,
}
```

```
impl Animal for Dog {
    fn new(name: String) -> Self {
        Dog {name}
    }
    fn say_hi(&self) {
        println!("dog named {self.name}");
    }
}
```

Trait example (bad)

```
fn introduce<T>(animal: T) {
    animal.say_hi();
}
```

Compiler error!
Not all Ts define .say_hi()!

Trait example (good)

```
fn introduce<T: Animal>(animal: T) {
    animal.say_hi();
}
```

Trait bound means that this function is only parameterized for T is an Animal

Trait Syntactic Sugar

```
fn introduce(animal: impl Animal) {
    animal.say_hi();
}
```

Complex trait bounds: the where clause

```
fn add_stuff<T, T2>(a: T, b: T) -> T2
where
    T: std::ops::Add<Output = T2> + Clone + Copy + Default,
    T2: std::cmp::PartialEq + From<f64> + Into<f64>,
{
   <function body>
}
```

Common Traits

- `T: Clone` means that `t.clone() -> T` is defined
- `T: Default` means that `T::default() -> T` is defined
- `T: ToString` means that `t.to_string() -> String` is defined
- `T: From<T2>` means that `T::from(t2: T2) -> T` is defined
  - It *implies* that `T2: Into<T>`, aka `T2::into(t: T) -> T` is defined "automatically"
- `T: Deref` overrides the unary * (dereference) operator
  - Box<T> behaves the same as &T
- `T: Display` means that `t.fmt()` is defined
  - `println!("{t}")` only works on `t: T` where `T: Display`!
- `T: Debug` is similar to display but is auto-generatable for many types
  - `println!("{t:?}")` is the syntax for debug printing `t`

Drop Trait

`T: Drop` means that `t.drop()` is defined.

The compiler calls this for you when a variable goes out of scope.

- Drop == running a destructor on the data
- You can manually drop a variable by using this function: (defined in std::mem)

```
pub fn drop<T>(_x: T) {}
```

Marker Traits

- `T: Copy` means that a copy of `t` can be made via a simple bit-level memcpy
  - No functions need to be implemented to be Copy!
  - Instead, Rust will check to make sure that all elements of `T` are also `Copy`
- `Sized`
  - The type has a known size at compile time (unlike str and dyn T)
- `Send, Sync`
  - The foundation for Rust's "fearless concurrency"
  - We will talk more about these marker traits later!

Derive

Many traits can be automatically derived (when possible), e.g.

```
#[derive(Debug, Copy, Clone)]
struct Foo {
    bar: u64,
    baz: i32,
}
```

Associated Types

Traits can define types that the implementer must override.

```
pub trait Animal {
    type Food;
    fn eat(&mut self, food: Self::Food);
}
impl Animal for Cow {
    type Food = Grass;
    fn eat(&mut self, food: Grass) {...}
}
```

Iterators

Let's put it all together with iterators!

`Iterator`: a trait that defines

```
fn next(&mut self) -> Option<Self::Item>;
```

Where `Self::Item` is an associated type for the `Iterator` trait.

When iterator is implemented for a type, Rust allows you to `for` loop over it!

Iterators

Efficiently implement functional-style programming:

```
let list = [
    Rectangle { width: 10, height: 1 },
    Rectangle { width: 3, height: 5 },
    Rectangle { width: 7, height: 12 },
];

list.iter().filter(|r| r.width == 3);
```

This is a *closure*

Quick Iterator Demo

We want this to work:

```
for num in (Counter {value: 4}) {
    println!("num is {num}")
}
```

A quick aside: Transferring ownership

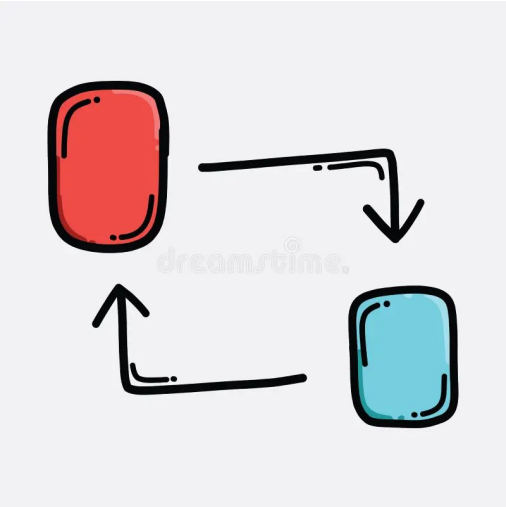
What if we want x and y to swap values?

There is a std::mem function for this! std::mem::swap

```
let mut x = 5;
let mut y = 42;

std::mem::swap(&mut x, &mut y);

assert_eq!(42, x);
assert_eq!(5, y);
```



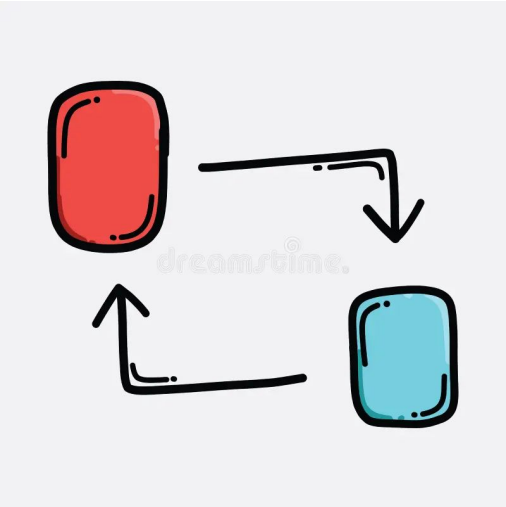
A quick aside: Transferring ownership

Why is this useful?

You can never have an undefined struct element in Rust!

```
let x = Rectangle {contents: vec![1,2]}
let y = Rectangle {contents: vec![3,4,5]}

// swap x and y's contents
let tmp = y.contents; // moved all of y!
// now y.contents is undefined
y.contents = x.contents; // cannot use y!
x.contents = tmp;
```



A quick aside: Transferring ownership

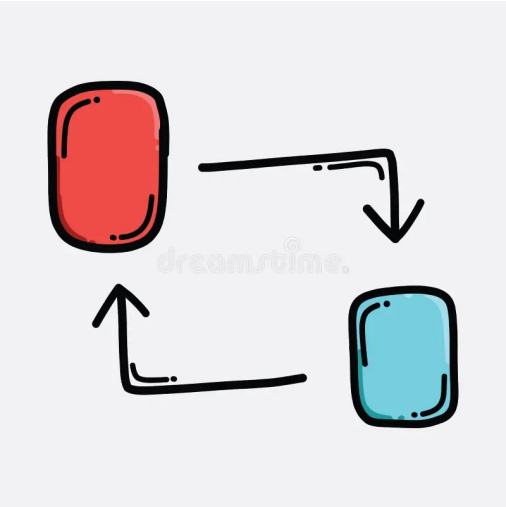
Why is this useful?

You can never have an undefined struct element in Rust!

```
let x = Rectangle {contents: vec![1,2]}
let y = Rectangle {contents: vec![3,4,5]}

// swap x and y's contents

std::mem::swap(&mut x.contents, &mut y.contents);
```



Moving on: Pointer Types

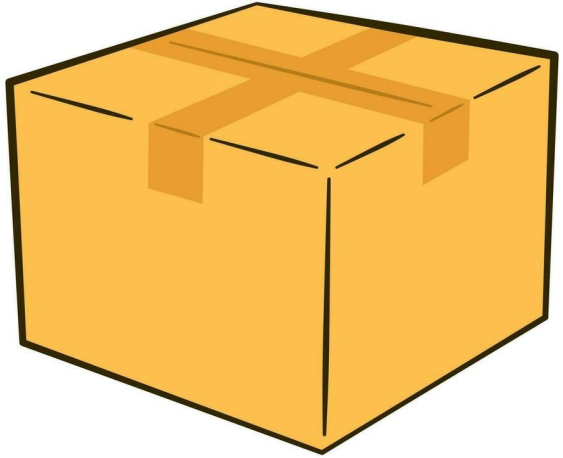
	Owned	Borrowed
Value	T	
Pointer		&T

Moving on: Pointer Types

	Owned	Borrowed
Value	T	
Pointer	Box<T>	&T

Box<T>

- T on the Heap
- Can obtain a mutable reference to inner value
- Still owned by a stack variable
  - So that the compiler knows when to free!



If a slice is a &[T]...

What is a Box<[T]>?

- Also a slice! But one that you own!

Demo

- Linked List (Box<T>)

Quiz (11:14-11:20)

Select all that apply.

```rust
struct Vec2 {
    x: f64,
    y: f64,
}
impl Vec2 {
    fn scale(self: Vec2, factor: f64) -> Vec2 {
        Vec2 {
            x: self.x * factor,
            y: self.y * factor,
        }
    }
}
fn main() {
    let v = Vec2 { x: -1.0, y: 5.0 };
    v.scale(2.0);
    println!("The upscaled Vec2 is: {} {}", v.x, v.y);
}
```